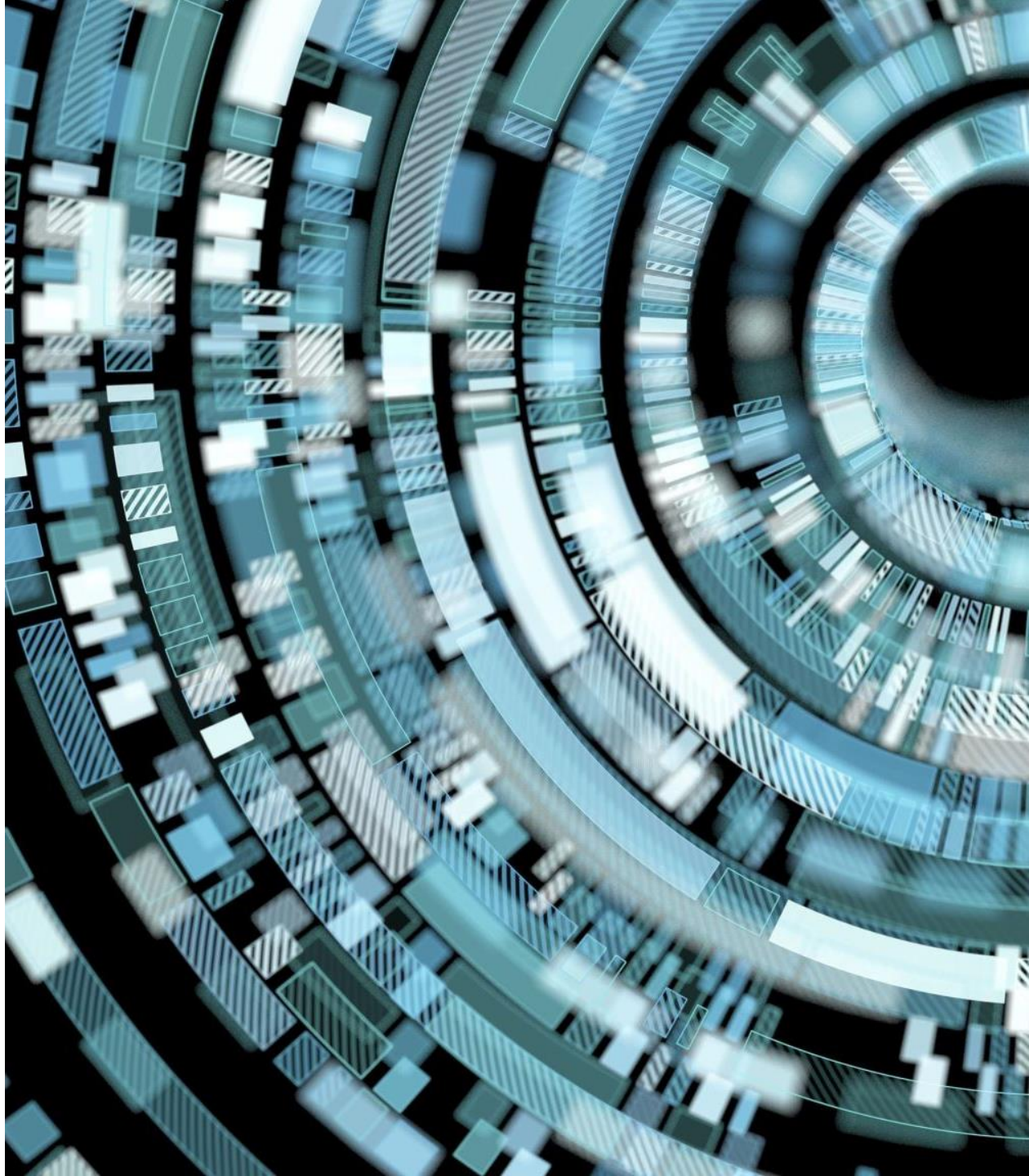
Visualisation



Overview

- Most common used chart Type
- Line plot
- Working with text and label
- Sub plot
- Bar plot
- Scatter plt
- Histogram
- Box plot
- Heat map

Selecting the Right Chart Type

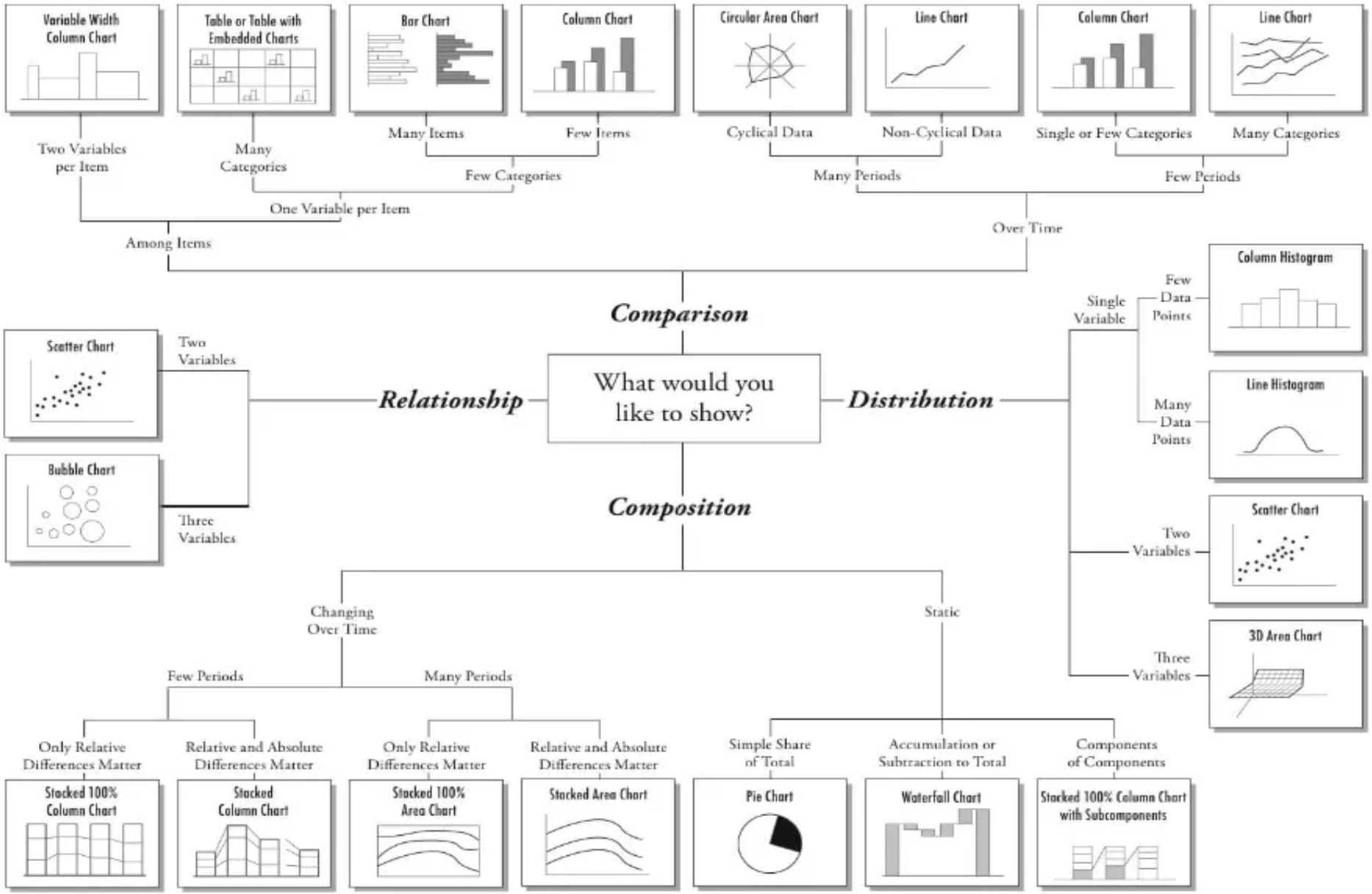
Which Chart is best depends upon:

- How many variables do you want to show in a single chart?
- How many data points will you display for each variable?
- Will you display values over a period of time, or among items or groups?

There are four basic presentation types:

- Comparison
- Composition
- Distribution
- Relationship

Chart Suggestions—A Thought-Starter



Most Common Chart Types

- Scatter Plot
- Histogram
- Bar & Stack Bar Chart
- Box Plot
- Pie Chart
- Heat Map

Popular Libraries of Python for Visualization

Matplotlib: low level, provides lots of freedom

Pandas Visualization: easy to use interface, built on Matplotlib

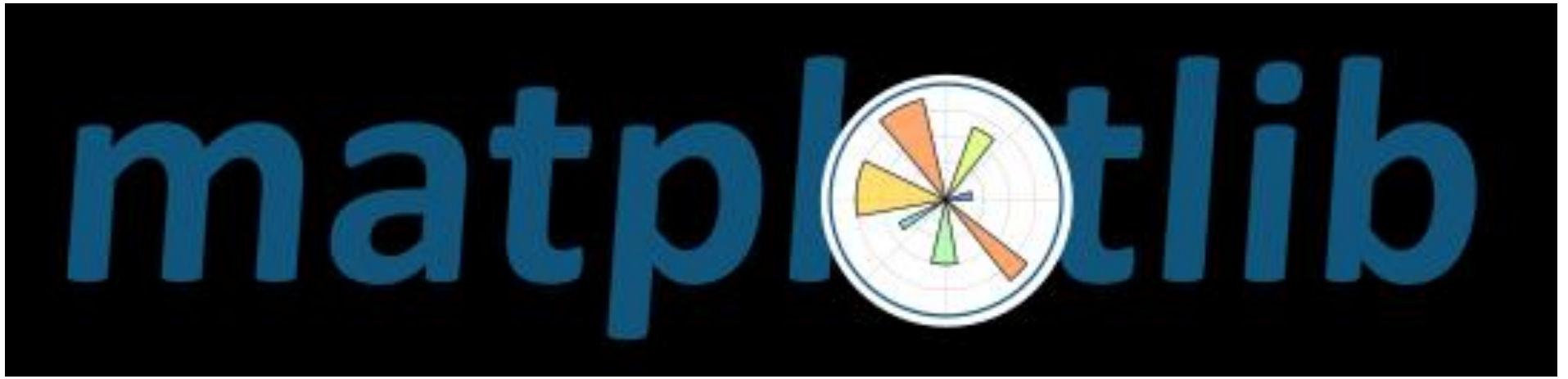
Seaborn: high-level interface, great default styles

ggplot: based on R's ggplot2, uses Grammar of Graphics

Plotly: can create interactive plots

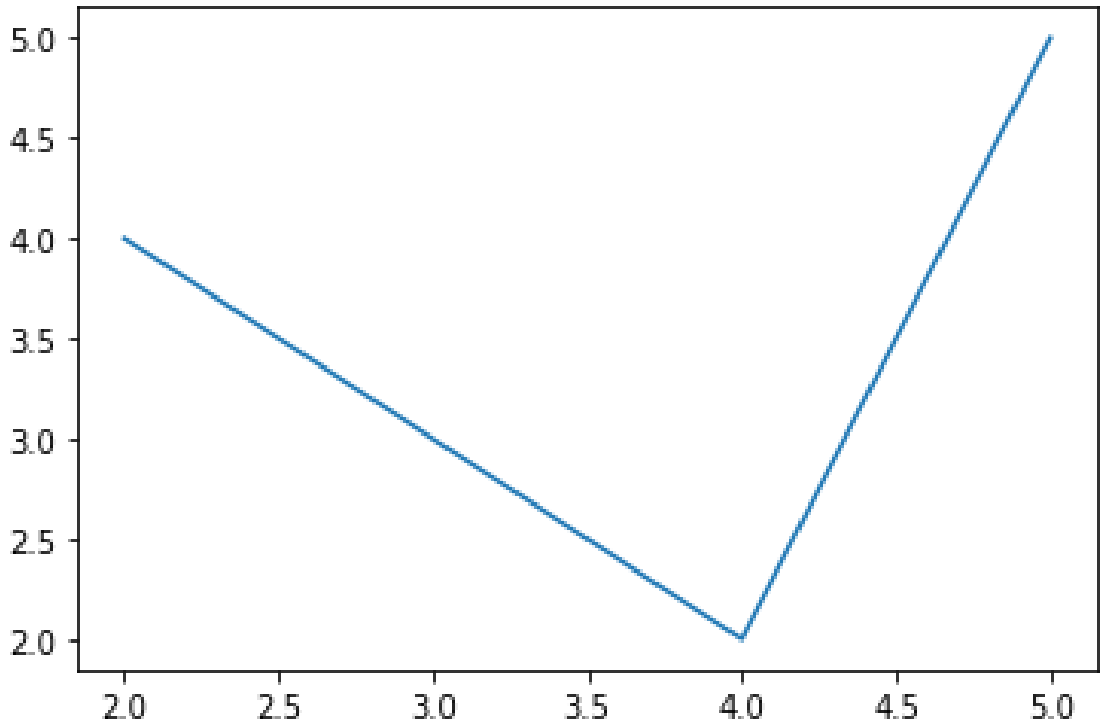
Examples of Visualization

Commonly used Plots using Matplotlib



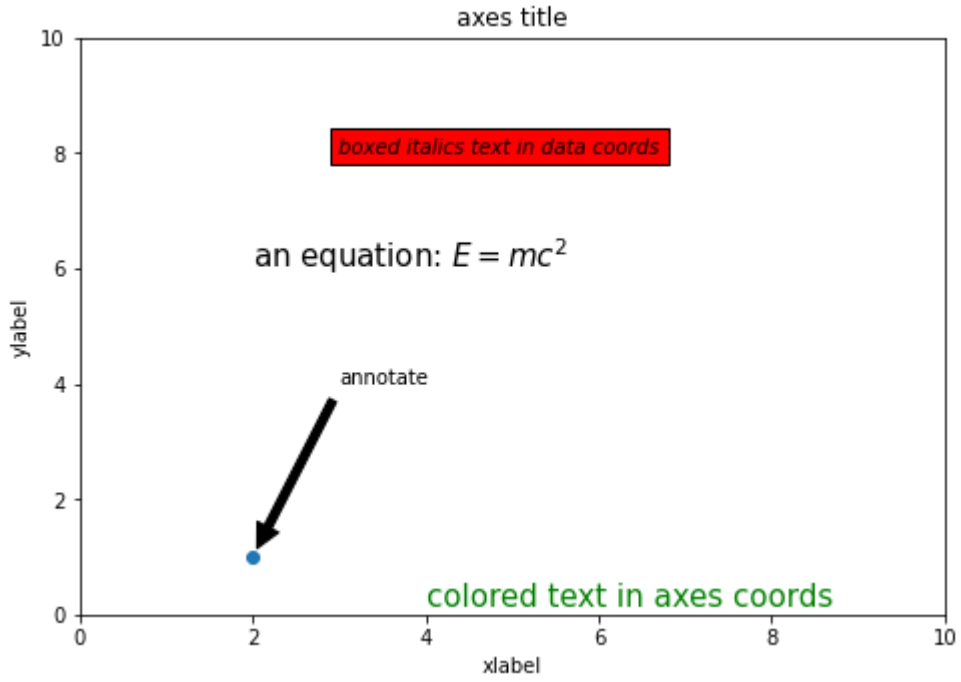
Line plot

- import matplotlib.pyplot as plt
- x=[2,3,4,5]
- y=[4,3,2,5]
- plt.plot(x,y)
- plt.show()



Matplotlib – Working With Text

- Matplotlib has extensive text support, including support for mathematical expressions, TrueType support for raster and vector outputs, newline separated text with arbitrary rotations, and unicode support. Matplotlib includes its own matplotlib.font_manager which implements a cross platform, W3C compliant font finding algorithm.



- aimport matplotlib.pyplot as plt
- fig = plt.figure()
- ax = fig.add_axes([0,0,1,1])
- ax.set_title('axes title')
- ax.set_xlabel('xlabel')
- ax.set_ylabel('ylabel')
- ax.text(3, 8, 'boxed italics text in data coords', style='italic',
- bbox={'facecolor': 'red'})
- ax.text(2, 6, r'an equation: $E=mc^2$', fontsize=15)
- ax.text(4, 0.05, 'colored text in axes coords',
- verticalalignment='bottom', color='green', fontsize=15)
- ax.plot([2], [1], 'o')
- ax.annotate('annotate', xy=(2, 1), xytext=(3, 4),
- arrowprops=dict(facecolor='black', shrink=0.05))
- ax.axis([0, 10, 0, 10])
- plt.show()

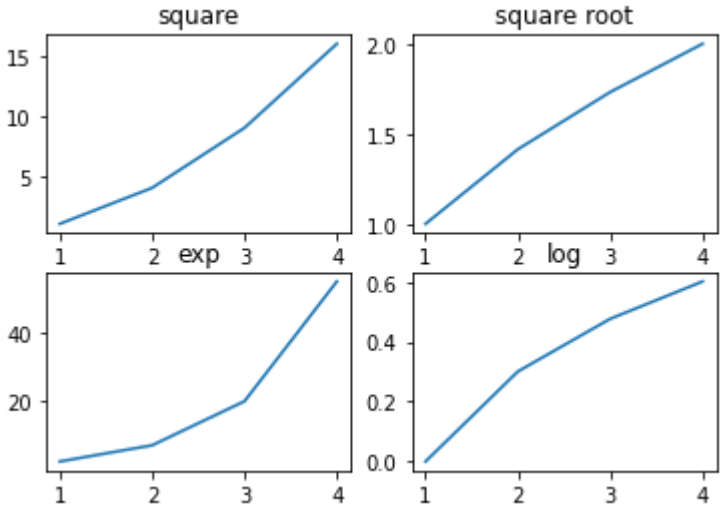
Matplotlib – Subplots() Function

Matplotlib’spyplot API has a convenience function called subplots() which acts as a utility wrapper and helps in creating common layouts of subplots, including the enclosing figure object, in a single call.

Plt.subplots(nrows, ncols)

The two integer arguments to this function specify the number of rows and columns of the subplot grid. The function returns a figure object and a tuple containing axes objects equal to nrows*ncols. Each axes object is accessible by its index. Here we create a subplot of 2 rows by 2 columns and display 4 different plots in each subplot.

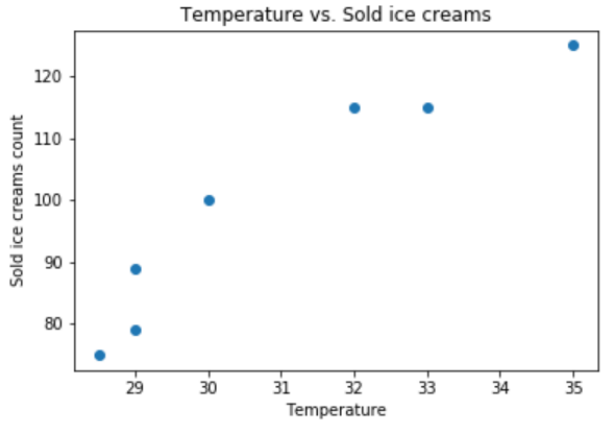
```
import matplotlib.pyplot as plt
fig,a= plt.subplots(2,2)
import numpy as np
x=np.arange(1,5)
a[0][0].plot(x,x*x)
a[0][0].set_title('square')
a[0][1].plot(x,np.sqrt(x))
a[0][1].set_title('square root')
a[1][0].plot(x,np.exp(x))
a[1][0].set_title('exp')
a[1][1].plot(x,np.log10(x))
a[1][1].set_title('log')
plt.show()
```



Scatter Plot

➢**When to use**: Scatter Plot is used to see the relationship between two continuous variables.

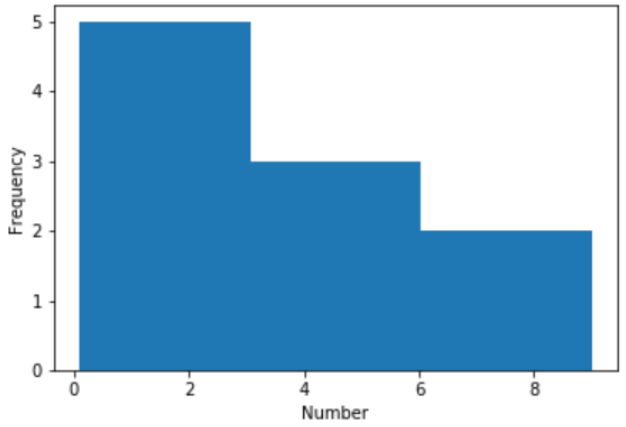
```
import matplotlib.pyplot as plt

temp = [30, 32, 33, 28.5, 35, 29, 29]
ice_creams_count = [100, 115, 115, 75, 125, 79, 89]

plt.scatter(temp, ice_creams_count)
plt.title("Temperature vs. Sold ice creams")
plt.xlabel("Temperature")
plt.ylabel("Sold ice creams count")
plt.show()
```



Histogram

➢**When to use**: Histogram is used to plot continuous variable. It breaks the data into bins and shows frequency distribution of these bins.

```
import matplotlib.pyplot as plt

numbers = [0.1, 0.5, 1, 1.5, 2, 4, 5.5, 6, 8, 9]

plt.hist(numbers, bins = 3)
plt.xlabel("Number")
plt.ylabel("Frequency")
plt.show()
```

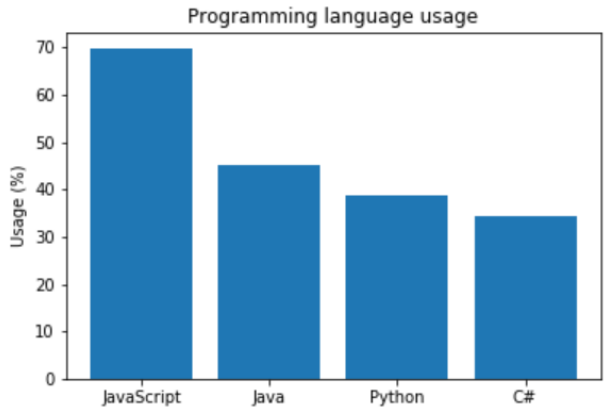


Simple Histrogram Ouput

Bar Chart

➢**When to use**: Bar charts are recommended when you want to plot a categorical variable or a combination of continuous and categorical variable.

```
labels = ["JavaScript", "Java", "Python", "C#"]
usage = [69.8, 45.3, 38.8, 34.4]

# Generating the y positions. Later, we'll use them to replace them with labels.
y_positions = range(len(labels))

# Creating our bar plot
plt.bar(y_positions, usage)
plt.xticks(y_positions, labels)
plt.ylabel("Usage (%)")
plt.title("Programming language usage")
plt.show()
```

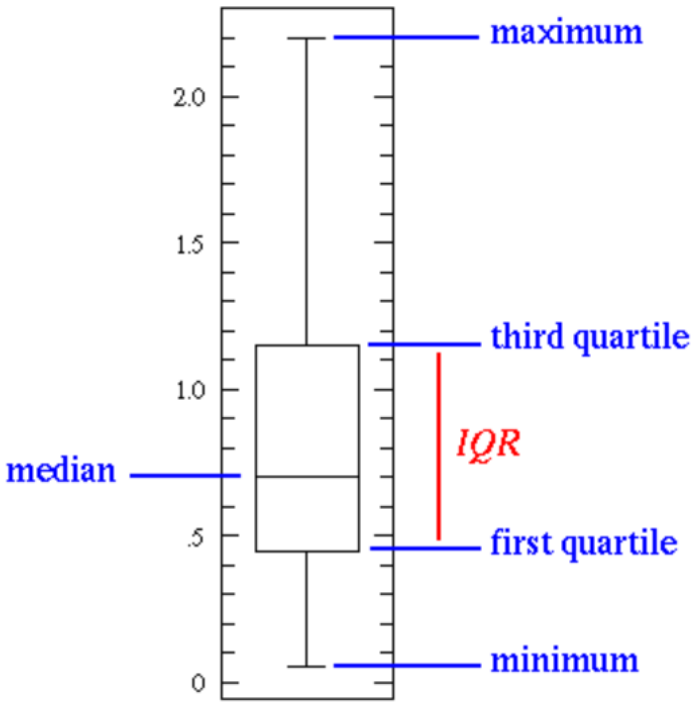


Simple Bar Chart Output

Boxplot

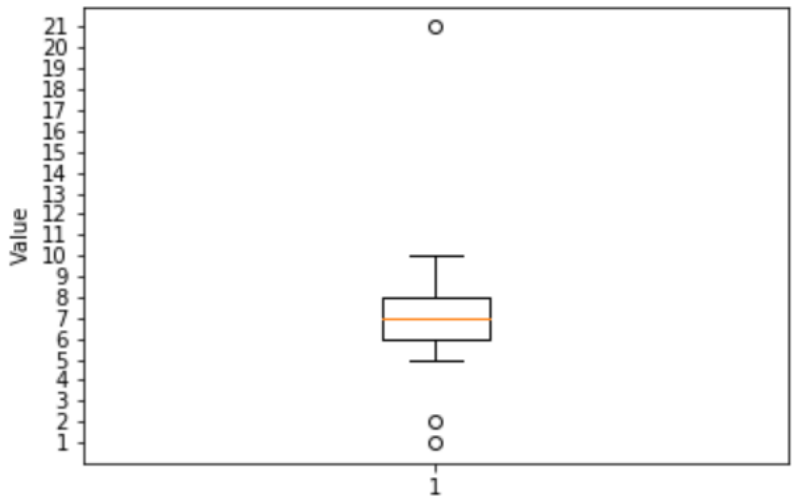
➢ **When to use**: Box Plots are used to plot a combination of categorical and continuous variables.

➢ This plot is useful for visualizing the spread of the data and detect outliers.



Box Plot. Source: http://www.physics.csbsju.edu/stats/box2.html

Boxplot



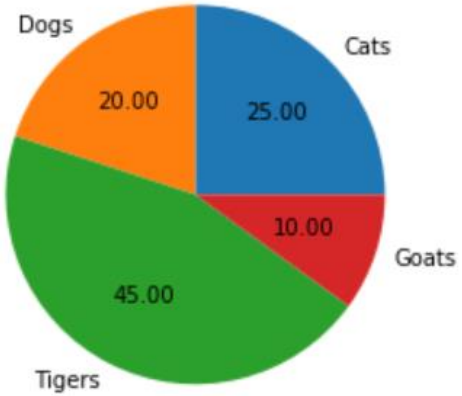
Simple Box Plot Output

```
values = [1, 2, 5, 6, 6, 7, 7, 8, 8, 8, 9, 10, 21]


plt.boxplot(values)
plt.yticks(range(1, 22))
plt.ylabel("Value")
plt.show()
```

Pie Chart

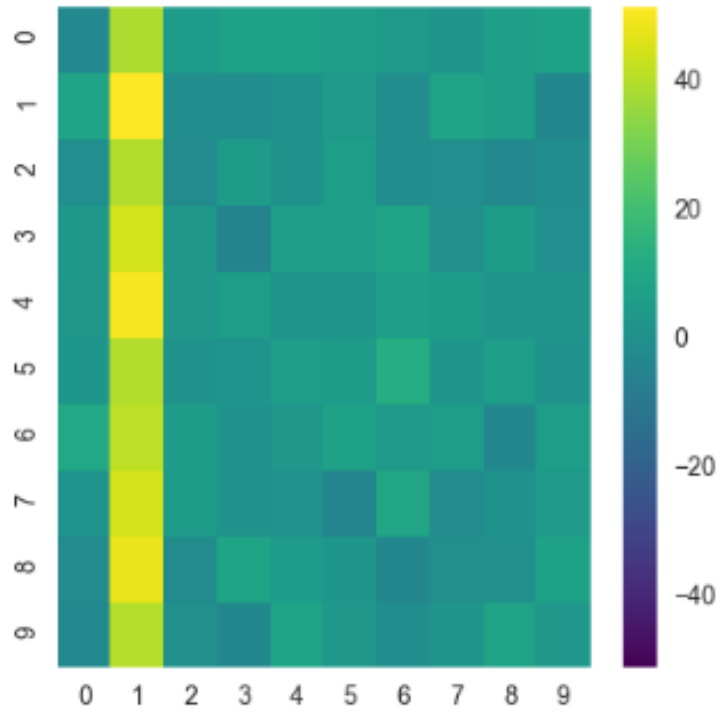
➢**When to use**: They are widely used in the **business world**. However, many experts recommend to **avoid them**. The main reason is that it’s difficult to compare the sections of a given pie chart.

```
sizes = [25, 20, 45, 10]

labels = ["Cats", "Dogs", "Tigers", "Goats"]


plt.pie(sizes, labels = labels, autopct = "%.2f")

plt.axes().set_aspect("equal")

plt.show()
```



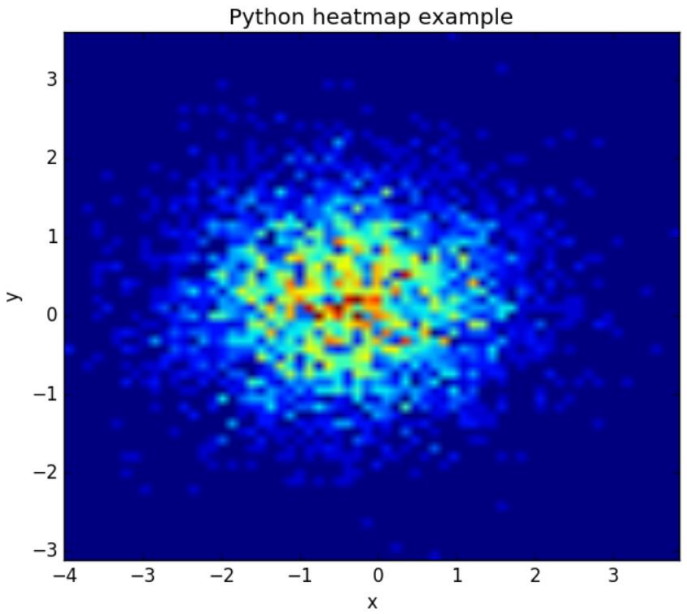
HeatMap

- A heat map (or heatmap) is a graphical representation of data where the individual values contained in a matrix are represented as colors.
- It is a bit like looking a data table from above.
- It is really useful to display a general view of numerical data, not to extract specific data point.



Heat Map

The histogram2d function can be used to generate a heatmap.




```
import numpy as np
import numpy.random
import matplotlib.pyplot as plt

# Create data
x = np.random.randn(4096)
y = np.random.randn(4096)

# Create heatmap
heatmap, xedges, yedges = np.histogram2d(x, y, bins=(64,64))
extent = [xedges[0], xedges[-1], yedges[0], yedges[-1]]

# Plot heatmap
plt.clf()
plt.title('Pythonspot.com heatmap example')
plt.ylabel('y')
plt.xlabel('x')
plt.imshow(heatmap, extent=extent)
plt.show()
```